Software Architecture Specification (SAS)

Team3-Basyx-Editor

Version Control

Version	Date	Author	Comment
1.0	29.10.2025	Florian Zahn	Initialize a first draft of the SAS
1.1	05.11.2025	Florian Zahn	Added Subsystem Specifications (MOD01–MOD04) derived from SRS/CRS use cases.
1.2	06.05.2026	Florian Zahn	Updated Architectural Concept and Subsystem Specifications according to updated SRS/CRS
1.3	15.05.2026	Florian Zahn	Updated MOD references in Subsystem Specifications

Table of Contents

1 Introduction

The purpose of this document is to describe the software architecture of the **BaSyx Editor Extension**, which builds upon the open-source **Eclipse BaSyx AAS Web UI**.
This project aims to extend the existing Asset Administration Shell (AAS) visualization and management interface by adding advanced **editing**, **validation**, and **plugin management** functionalities.

Key quality attributes addressed: *extensibility*, *maintainability*, *interoperability*, and *usability*.

2 Scope

The BaSyx Editor Extension enhances the capabilities of the BaSyx Web UI by enabling users to create, modify, and validate AAS models directly through the web interface.
It also introduces a structured plugin system for specialized submodel visualization and editing components.
The software integrates with existing BaSyx backend services, such as the **AAS Registry**, **AAS Repository**, and **Submodel Service**.

3 System Overview

3.1 System Environment

The system operates as a **web-based client application** interacting with BaSyx backend services.
It communicates over HTTPS and uses the AAS REST API (V3).
The following external components are required:

- BaSyx Registry Service
- BaSyx AAS Repository
- BaSyx Submodel Service

3.2 Software Environment

- **Frontend**: Vue.js 3, TypeScript, Vite build system
- **Backend (optional)**: Node.js-based proxy or BaSyx Java SDK V2
- **Deployment**: Docker container or static web hosting
- **Version Control**: Git / GitHub

4 Architecture

4.1 Architectural Concept

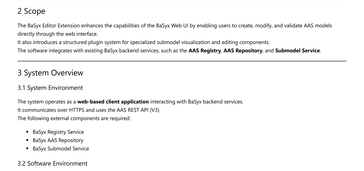

Figure 4.1: High-level System Context Diagram

The BaSyx Editor Extension acts as an **interactive frontend** that connects to the existing BaSyx backend ecosystem.
It leverages BaSyx's REST APIs to query, update, and validate AAS structures.

4.2 Architectural Model

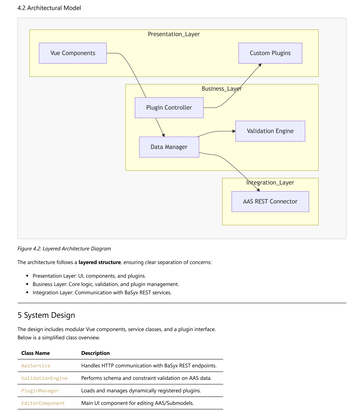

Figure 4.2: Layered Architecture Diagram

The architecture follows a **layered structure**, ensuring clear separation of concerns:

- Presentation Layer: UI, components, and plugins.
- Business Layer: Core logic, validation, and plugin management.
- Integration Layer: Communication with BaSyx REST services.

5 System Design

The design includes modular Vue components, service classes, and a plugin interface.
Below is a simplified class overview.

Class Name	Description
`AasService`	Handles HTTP communication with BaSyx REST endpoints.
`ValidationEngine`	Performs schema and constraint validation on AAS data.
`PluginManager`	Loads and manages dynamically registered plugins.
`EditorComponent`	Main UI component for editing AAS/Submodels.
`SubmodelVisualizer`	Displays specific submodel types with custom components.

6 Product Interfaces

6.1 User Interfaces

The user interacts via a browser-based graphical interface built with Vue.js.
The UI provides forms, trees, and visual editors for AAS structures.

6.2 Software Interfaces

- REST communication via Axios (HTTP client).
- Plugin system exposed through a JavaScript interface (`IPlugin`).
- Modules communicate using event emitters and reactive stores.

6.3 Communication Interfaces

- Protocol: HTTPS
- Format: JSON / AAS Metamodel V3 compliant
- REST Endpoints:
  - `/registry/*`
  - `/submodels/*`
  - `/shells/*`

7 Subsystem Specification

7.1 MOD01 – File Import and Validation Subsystem

Field	Description
Subsystem ID	MOD01
Related Use Cases	UC01
Covered Requirements	FR.01 (MimeType Detection of Model Files), FR.02 (Plausibility Check), FR.03 (Readable Error Message After Plausibility Check), NFR.01 (Usability), NFR.05 (Compatibility), NFR.06 (Error Handling)
Service	Provides import functionality for external model files such as KBL, VEC, XML and AML. The subsystem detects the correct MimeType, performs a plausibility check between file extension and content structure, and displays a readable error message if validation fails. After successful validation, the file is made available for further processing and can be linked in the AAS context.
Interfaces	UI file import component; `BaSyxSubmodelREST API`; internal `MimeTypeDetectionService`; internal `FileUtilityOtherUserScriptForMessageComponent`
Postcondition	The file is validated, the MimeType is assigned, and the file can be used by later processing steps such as XML viewing or AAS generation.
Module Documentation	MOD-UC1

7.2 MOD02 – XML Viewer and Navigation Subsystem

Field	Description
Subsystem ID	MOD02
Related Use Cases	UC02
Covered Requirements	FR.04 (Table of Contents in XML Visualization), NFR.01 (Usability), NFR.02 (Performance), NFR.05 (Compatibility)
Service	Provides a Vue-based XML viewer component for displaying imported XML-based files such as KBL, VEC or AML. The subsystem builds a table of contents from the XML structure and allows the user to navigate to specific sections and inspect node details, attributes and IDs.
Interfaces	UI component `XmlViewerComponent`; internal `XmlTocBuilder`; backend endpoint for retrieving file content; BaSyx AAS/Submodel REST API
Postcondition	The XML content is displayed in a structured and navigable way. The user can inspect relevant XML sections without manually searching through the complete file.
Module Documentation	MOD-UC2

7.3 MOD03 – AAS Generation and Submodel Mapping Subsystem

Field	Description
Subsystem ID	MOD03
Related Use Cases	UC03
Covered Requirements	FR.05 (AAS Generator Wizard and Property Adoption), FR.06 (Background Processing for KBL/VEC to AAS Submodels), NFR.02 (Performance), NFR.03 (Maintainability), NFR.05 (Compatibility), NFR.06 (Error Handling)
Service	Implements the wizard-driven generation of an Asset Administration Shell from KBL/VEC files. The user selects the relevant properties in the wizard. In the background, the client parses the KBL/VEC file and transforms the selected data into valid AAS submodels and submodel elements according to defined mapping rules. The generated structure includes the submodels `HandoverDocumentation` with the original file attached and `TechnicalData` with the extracted data points. Only validated and correctly generated submodels are sent to the backend.
Interfaces	UI component `AasGeneratorWizard`; internal `KblVecParser`; internal `SubmodelMappingService`; REST endpoint `/aas/generate`; REST endpoint `/submodels/generate`; BaSyx AAS/Submodel REST API
Postcondition	A new AAS is created and stored on the AAS Server. The submodels `HandoverDocumentation` and `TechnicalData` exist and contain the expected file reference and extracted data points.
Module Documentation	MOD-UC3

7.4 MOD04 – Targeted Data Retrieval Subsystem

Field	Description
Subsystem ID	MOD04
Related Use Cases	UC04
Covered Requirements	FR.07 (REST API for Targeted Data Retrieval), NFR.02 (Performance), NFR.06 (Error Handling)
Service	Provides a backend REST API for inspecting and querying external structured files linked in the BaSyx context. The subsystem supports XML and JSON source files that are accessible via HTTP. It can inspect the internal structure of a file, such as an XML tree or JSON hierarchy, and retrieve selected elements by path, ID or field. The response payload is always returned as XML and contains only the requested element or value, not the complete source document.
Interfaces	API endpoint `/api/inspect`; API endpoint `/api/query`; external HTTP file reference; BaSyx AAS Repository Service for resolving linked file content
Postcondition	The requesting API client receives only the requested data point(s) as XML. Invalid file references or query paths are handled with clear client errors such as HTTP 400 or 404.
Module Documentation	UC04

8 Technical Concept

8.1 Frontend Architecture

The frontend is built with Vue.js and Vite. It follows a modular structure using the Composition API.
Components are grouped by feature and dynamically imported for performance.

8.2 Plugin Integration

Plugins are registered by adding them to the `UserPlugins` directory.
Each plugin implements the `IPlugin` interface to define entry points and hooks into the editor lifecycle.

8.3 Data Validation & Synchronization

The validation engine ensures that any modified AAS structure adheres to the AAS Metamodel v3.
Synchronization mechanisms guarantee data consistency between client and repository.

8.4 Deployment Strategy

The application is deployed as a *Docker container* or static web application.
Environment variables configure URLs of the BaSyx Registry and Repository services.

8.5 Error Handling

Errors are handled via global interceptors and user notifications.
Validation and network errors are displayed in the UI with context-specific messages.

9 References

[1] Eclipse BaSyx AAS Web UI Repository – https://github.com/eclipse-basyx/basyx-aas-web-ui
[2] AAS Metamodel Specification V3 – https://industrialdigitaltwin.org/
[3] Vue.js Documentation – https://vuejs.org/
[4] BaSyx Wiki – https://wiki.basyx.org/